2025 IEEE 11th International Conference on Big Data Computing Service and Machine Learning Applications (BigDataService)

2025 IEEE 11th International Conference on Big Data Computing Service and Machine Learning Applications (BigDataService) | 979-8-3315-8532-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/BIGDATASERVICE65758.2025.00020

Deep Learning Enhanced Load Balancing with NeuroPointSelect for Edge Computing

Yeswanth Surya Srikar Nuchu
Software Engineer II
Dallas, Texas
https://orcid.org/0000-0002-6681-2314

Devi Navya Nuchu
Senior Software Engineer
Austin, Texas
devinavya.nuchu@gmail.com

***Abstract*—Compute-intensive instances require load balancing together with timely auto-scaling to function effectively under systems with changing resource utilization patterns. The traditional load-balancing approaches fail to adjust to changing workload patterns because they separate load balancing from auto-scaling into distinct services which leads to delayed scaling responses and poor request placement and excessive resource allocation. NeuroPointSelect presents a new deep learning algorithm which unifies resource-aware load balancing with proactive auto-scaling functions. The system assesses request resources through CPU and GPU and RAM evaluations and uses server telemetry data to direct requests to optimal servers while forecasting server capacity requirements to automate scaling actions. The proposed technique shows better performance than standard load-balancing approaches through its ability to optimize resource utilization and adaptability.**

Keywords—Deep Learning, Load Balancer, Auto Scaling.

I. Introduction

Load balancing plays a fundamental role in current distributed systems because it enables efficient resource use alongside system responsiveness. Network traffic distribution across multiple server's functions as load balancing to stop server overload while maximizing resource usage potential. The efficient distribution of workload through load balancing enables high system availability and reliability and supports scalability in extensive computing systems.

A. Auto Scaling and Its Role in Load Balancing

Real-time traffic demands influence auto scaling to dynamically change the number of operational servers. Auto scaling introduces more servers to handle traffic during load increases before it reduces server numbers to cut operational costs when traffic decreases. Both load balancing and auto scaling techniques work together for maximum resource optimization and smooth system operation.

B. Current Load Balancer Techniques

The main two types of load balancing algorithms exist under the categories of Static Load Balancing and Dynamic Load Balancing.

1. Static Load Balancing Techniques:

The distribution of traffic across servers in static load balancing relies on pre-defined rules or algorithms. These algorithms demonstrate high efficiency and predictability while functioning well in situations where workload patterns remain steady. Real-time traffic changes create challenges for these systems since they do not adapt easily.

Round Robin: This algorithm directs requests to servers in sequential order through a circular pattern. The approach divides network requests evenly between servers but fails to provide suitable performance when server processing power varies.

Weighted Round Robin: This method operates similarly to Round Robin but uses server capacity weights for distribution. The system directs more requests toward servers that hold higher weights. This method works well for systems that contain servers with different capacity levels because of its ability to distribute traffic accordingly.

The **IP hashing algorithm** uses the client's IP address to generate a hash value which determines the server for request assignment. The approach preserves session continuity and functions well for applications that maintain state.

2. Dynamic load balancing techniques:

The algorithms of dynamic load balancing continuously check server status for real-time workload information to perform traffic distribution. These algorithms provide better workload adaptability but require more computational power for their operation.

Least Connections: Requests are directed to the server which currently maintains the least number of active connections. This approach provides optimal performance when connection periods are inconsistent.

Weighted Least Connections operates similarly to Least Connections but incorporates server capacity weights to optimize resource distribution.

The **Least Response Time algorithm** directs requests toward servers that require the shortest time to respond. This algorithm stands out for applications that need fast responses and minimal latency because it provides optimal results.

The **adaptive algorithms** utilize real-time telemetry data about CPU and memory and network bandwidth to perform intelligent routing decisions.

Predictive Load Balancing makes use of machine learning models to analyze historical data which helps forecast traffic patterns and pre-assign requests. This method provides outstanding results for applications which experience variable workload patterns.

2690-828X/25/$31.00 ©2025 IEEE
DOI 10.1109/BigDataService65758.2025.00020

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

C. Deep Learning in Load Balancing

Modern deep learning techniques have established promising approaches which enhance load balancing operations. Real-time traffic routing decisions become possible through neural networks which process system telemetry information to detect anomalies while predicting future patterns. Neural networks process extensive historical and real-time data sets to find patterns while forecasting future system demands.

The system behavior enables reinforcement learning algorithms to continuously learn new strategies for load balancing. Deep learning models analyze server telemetry data to predict resource usage and take proactive steps in real-time to prevent congestion. This results in reduced latency, improved application performance, and optimized resource utilization.

D. Motivation for NeuroPointSelect

The current load balancing algorithms show weaknesses when handling dynamic workloads particularly in edge computing systems despite their progress. NeuroPointSelect emerged as a new load balancing solution because current methods needed an adaptive intelligent and efficient approach. NeuroPointSelect uses real-time telemetry analysis with deep learning to overcome conventional approach limitations while delivering optimized resource management and performance enhancement.

II. Literature Survey

Soussia et al. [1] demonstrates a load balancer that uses reinforcement learning to manage containerized workloads in Kubernetes environments dynamically. Through ongoing historical data analysis, the system generates more accurate distribution decisions. The system's decision-making depends mostly on past patterns which results in poor performance when dealing with unexpected traffic fluctuations or unpredictable workload variations. The model requires extended time to converge which results in occasional resource underutilization or overload problems.

Jyothi et al. [2] demonstrates reinforcement learning methods to maximize cloud load balancing effectiveness through analysis of Google Cluster data. The adaptive system provides enhanced resource efficiency together with lower operational expenses. The method shows benefits but its dependence on historical workload data makes it less responsive to current workload modifications. The system's inability to predict upcoming workload increases causes delayed scaling decisions which produces performance degradation when the system reaches peak usage.

Chaflekar et al. [3, 4, 5] introduces a model-agnostic predictive load balancer and auto scaling specifically designed for microservices-based applications. The system employs LSTM and GRU machine learning models to forecast future loads from access logs which leads to improved resource management. The predictive feature improves system performance, yet the weighted round-robin routing algorithm produces less than optimal resource distribution. The system faces difficulties in handling dynamic workload patterns because it does not contain adaptive feedback mechanisms.

Benifa et al. proposes RLPAS, a proactive auto-scaling algorithm using Reinforcement Learning (RL), specifically the SARSA algorithm, to address dynamic resource provisioning challenges in cloud environments [6]. Unlike traditional reactive or rule-based scaling methods, RLPAS uses parallel learning agents and function approximation to rapidly learn optimal resource allocation policies, thus minimizing response time and maximizing throughput and CPU utilization. RLPAS is validated on real-world workloads like RUBiS, RUBBoS, and Olio, and it outperforms existing methods by achieving faster convergence and better adaptability to workload fluctuations, while reducing resource wastage and ensuring SLA compliance.

III. Existing System

A. Existing Load Balancer Architecture

The base system operates with application servers that register with the load balancer when they start as shown in Fig 1. The load balancer directs incoming requests through fundamental methods including round robin distribution and least connections distribution and resource-based routing which selects servers according to total CPU and memory usage statistics. The straightforward methods provide easy implementation, but they cannot track how individual servers perform and what their current workload looks like.

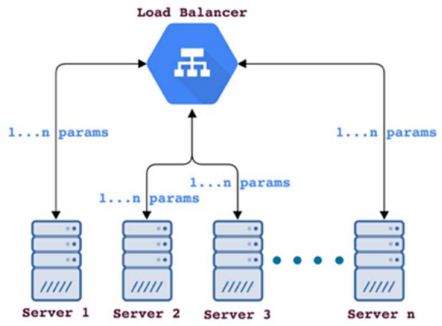

Fig. 1. Base System

B. Auto-Scaling as a Separate System

The present auto-scaling mechanism functions as an independent system which tracks complete application server resources including CPU and memory utilization. The system triggers scaling events after CPU utilization averages exceed 80% for five consecutive minutes. The system responds with delayed adjustments that lead to performance degradation when traffic suddenly increases. Transient demand spikes cause the system to scale up unnecessarily which results in wasted resources and higher operational expenses.

C. Common Problem

Service availability decreases because of two principal factors. First the auto scaling service does not provide proactive instance scaling functionality. Second the load

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

balancer algorithm lacks sufficient intelligence to direct traffic toward available servers that have sufficient resources to meet demands.

The following example shows how inadequate load balancer operations can negatively affect service availability.

The system contains 2 instances of service along with a load balancer that employs Adaptive/Resource-based Load Balancing to manage traffic distribution. The CPU usage of Instance 1 stands at 70% while Instance 2 uses 40% of its available resources. During this scenario the load balancer processed two requests received from the public end point (or clients). The first request needed 30% CPU while the second request needed 50% CPU. The load balancer directs request 1 to Instance 1 because request 1 needs more resources than request 2 will get. The system fails to process request 2 because Instance 2 lacks sufficient resources needed to handle the request as shown in Fig 2. A smart load balancer would assess the current state of the system and sends request 1 to server 2 and request 2 to server 1 to help maintain better availability.

A poorly designed auto-scaling system makes the problem worse because it takes too long to create new instances which leaves the system without sufficient capacity to process requests thus causing service degradation.

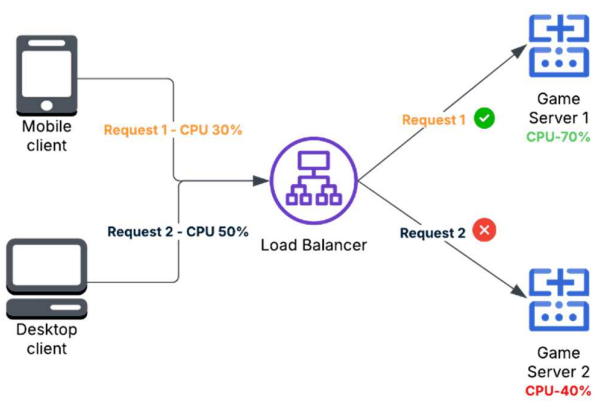

Fig. 2. Common problem

IV. Proposed System

A. Architecture Overview

As shown in Fig 3, the proposed system will operate in two steps:

Step 1: Metric Collection and Prediction using LSTM

The base architecture of the proposed system is the same as shown in Fig 1, where each app server sends its metrics to the load balancer every δt seconds. These metrics are received at the private metrics endpoint of the load balancer and stored in a circular buffer of length δl.

The circular buffer for each app server i is defined as:

Formula:

$$B_i(t) = \{M_i(t - k \cdot \delta t) \mid k = 0,1,\ldots,\delta x - 1\} \quad (1)$$

where:

- $M_i(t)$represents the metrics (CPU, GPU, RAM) received from server i at time t
- δx is the number of recent metrics stored per server

The metrics are mapped to each app server based on its IP address. This circular buffer stores the last δx metric sets for each server. The mapped data is then sent to a Long Short-Term Memory (LSTM) model [7] to predict the next metric array for the upcoming δt seconds.

$$\hat{M}_i(t + \delta t) = LSTM(B_i(t))\} \quad (2)$$

With this prediction, the load balancer has a forecasted metrics array for the next δt seconds, which helps it make decisions in Step 2.

Step 2: Resource Allocation and Auto-Scaling Decision

Client sends a request with resource parameters such as CPU, GPU, and RAM to the load balancer at the public endpoint.

$$R = (R_{CPU}, R_{GPU}, R_{RAM}) \quad (3)$$

With this prediction, the load balancer has a forecasted metrics array for the next δt seconds, which helps it make decisions in Step 2.

Then the load balancer passes this request information to a Randomized Quick Select algorithm to identify the index at which the resource set $(R_{CPU}, R_{GPU}, R_{RAM})$ falls within the predicted metrics array.

$$\hat{M}_i^{(I)}(\mathrm{t} + \delta \mathrm{t}) \approx \mathrm{R} \quad (4)$$

where:

- I is the index identified by the Quick Select algorithm.

Based on this index, the load balancer decides whether to trigger auto-scaling.

$$Trigger\ Auto\ Scaling\ if\ \max\left(\hat{M}_i^{(I)}(t + \delta t) - R\right) > \epsilon \quad (5)$$

where:

- ϵ is threshold parameter that indicates when to trigger scaling.

All values to the left of the identified index are smaller than the requested resources and are ignored. The elements greater than the identified index exceed the requested resources, so a Closest Point Selection algorithm is applied to this subset to determine the final IP address of the server.

$$S_i = arg\ min_{\widehat{\{j \in S\}}} || \hat{M}_j(t + \delta t) - R || \quad (6)$$

where:

- S is the set of servers with sufficient resources.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

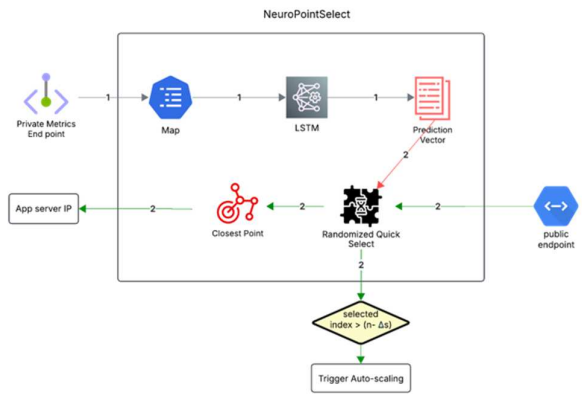

Fig. 3. Target System

B. Randomized Quick Select

A Randomized Quick Select algorithm operates on Manhattan distance-based transformed multi-dimensional resource predictions to filter the high-capacity servers from a pool of n servers form client request.

1. Problem Setup:

A client request defines the necessary resources needed for video streaming through the following parameters:

$$(R_{CPU}, R_{GPU}, R_{RAM}) \tag{7}$$

In Step 1 we have available predictions about resource availability across each server during the upcoming delta t seconds. For Server S_i, the predicted average availability over the interval is:

$$C_i = (C^i_{CPU}, C^i_{GPU}, C^i_{RAM}) \tag{8}$$

A Randomized Quick Select algorithm operates on Manhattan distance-based transformed multi-dimensional resource predictions to filter the high-capacity servers from a pool of n servers form client request.

We need to find the split point in the list of servers which satisfies the following conditions:

- All servers to the left of the split point have lower cumulative resource availability than the request.
- All servers to the right have higher availability.

2. Dimensionality Reduction:

We convert the 3D resource vector into a scalar while maintaining order by calculating the Manhattan distance from the origin for both the request and each server:

$$RequestMagnitude = \left|\left|(R_{CPU}, R_{GPU}, R_{RAM}) - (0,0,0)\right|\right| = (R_{CPU} + R_{GPU} + R_{RAM}) \tag{9}$$

$$ServerMagnitude_i = \left|\left|(C^i_{CPU}, C^i_{GPU}, C^i_{RAM}) - (0,0,0)\right|\right| = (C^i_{CPU} + C^i_{GPU} + C^i_{RAM}) \tag{10}$$

This converts the 3D problem into a 1D array of scalar values:

$$MagnitudeArray = [RequestMagnitude, ServerMagnitude_1 \ldots ServerMagnitude_n] \tag{11}$$

3. Applying Randomized Quick Select:

Randomized Quick Select operates on this array to locate the rank position of RequestMagnitude among sorted scalar values. The algorithm partitions the array randomly to move values below the pivot to the left side and values above the pivot to the right side. The partitioning process keeps track of RequestMagnitude at index 0 throughout the operation.

After partitioning finishes RequestMagnitude ends up at an index which separates values that are less than it from values that are greater.

The resulting index divides servers between those with lower capability and those with higher capability relative to the client request. Fig 4 will illustrate this flow.

4. Tracking the Index of RequestMagnitude

As Quick Select partitions the array in-place:

- We insert RequestMagnitude at the beginning of the array.
- We track its position throughout the algorithm.
- Once RequestMagnitude reaches its final position k, we know:

$$\forall i < k,\ \text{MagnitudeArray}_i < \text{RequestMagnitude}$$

$$\forall i > k,\ \text{MagnitudeArray}_i > \text{RequestMagnitude}$$

The index k functions as the threshold index which determines server separation based on their capacity to process requested loads.

In Fig 4, explains the Randomized Quick Select with an example. Request $R1(50, 51, 40)$ is received at the public end point. This request is appended to the forecasted metrics array and applied dimensionality reduction technique to generate magnitude array [141, 139, 168, 143]. Randomized Quick Select is applied on the generated magnitude array to identify the request magnitude index, which is 1 in this example.

Time complexity $T(n)$ for the Randomized Quick Select is $O(n)$

- Appending to the forecasted metrics, which is appending the request R1 at the end and swapping with index 0 is $O(1)$.
- Dimensionality reduction technique, which is calculating the distance from origin (0,0,0) to generate the magnitude array is $O(n)$.
- Applying Randomized Quick Select on the generated magnitude array takes $O(n)$.

So, the overall time complexity is $O(n)$

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

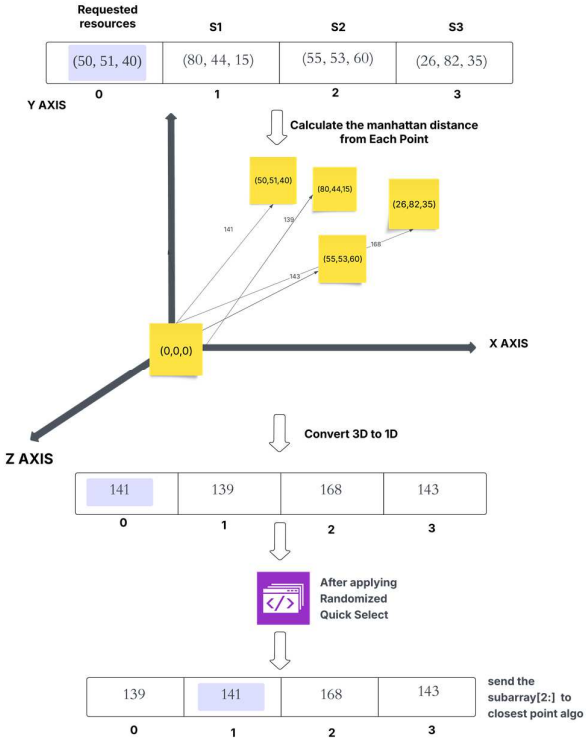

Fig. 4. Randomized Quick Select

C. Proactive Auto-Scaling

After Randomized Quick Select places the RequestMagnitude at its proper position the load balancer uses this index to decide if auto-scaling should start. The scalarized 1D representation of server resource availability shows all values left of the RequestMagnitude index as servers with insufficient capacity to handle the request since they fall below the required CPU, GPU, and RAM thresholds and are therefore excluded from the candidate pool. The servers whose predicted capacities exceed the resource requirements are located to the right of the index and are eligible to serve the request.

The position of RequestMagnitude within the array provides a real time indicator of system load. If the index is located near the end of the array, it means that only a few servers have the required capacity to meet the request. This is a predictive trigger for scaling and indicates potential saturation of system resources. In such cases, the load balancer will trigger a proactive auto-scaling event and provision additional server instances in anticipation of continued high demand conditions. This strategy enables the system to maintain service availability and prevent performance degradation.

The method differs from traditional reactive scaling approaches that depend on lagging indicators such as average utilization or queued requests because it enables forward-looking request-specific auto-scaling. The system enhances responsiveness through tight integration of server selection and capacity forecasting into a unified decision framework which optimizes computational resource utilization during dynamic workload conditions.

D. Algorithm and Pseudo Code

Step 1: Metric Collection and Prediction using LSTM

```
Input:
  δt → Time interval for metric collection
  δl → Circular buffer length
  δx → Number of recent metric sets to store
  AppServers[] → List of application servers

Initialize:
  CircularBuffer[AppServer] ← Empty buffer of size δl for each
AppServer

While True:
  For each AppServer in AppServers:
    Metrics ← CollectMetrics(AppServer)
    Append Metrics to CircularBuffer[AppServer]

    If Size (CircularBuffer[AppServer]) ≥ δx:
      InputData ← GetLastXMetrics(CircularBuffer[AppServer], δx)
      PredictedMetrics ← LSTMModel(InputData)
      Store PredictedMetrics for AppServer
  Wait δt seconds
```

Step 2: Resource Request Handling and Auto-Scaling Decision

```
Input:
  R = (R_CPU, R_GPU, R_RAM) → Resource request from client
  ε → Threshold for auto-scaling decision
  PredictedMetrics[] → Latest predicted metrics from Step 1

Function HandleResourceRequest(R):
  AllMetrics ← []

  For each AppServer in AppServers:
    AllMetrics.append(PredictedMetrics[AppServer])

  # Step 2.1: Perform Randomized Quick Select
  Index ← QuickSelect(AllMetrics, R)

  # Step 2.2: Evaluate Auto-Scaling Condition
  If Max(AllMetrics[Index] - R) > ε:
    TriggerAutoScaling()

  # Step 2.3: Apply Closest Point Selection
  CandidateMetrics ← AllMetrics[Index+1 : ]
  BestServer ← ClosestPointSelection(CandidateMetrics, R)

  Return BestServer.IP
```

V. Results

A. Simulation Environment:

A custom-built simulation environment performed the performance evaluation of Weighted Round Robin, Least Connections, Weighted Least, and NeuroPointSelect algorithms. Real-time load balancing operations are emulated by the simulator through its replication of client requests which specify CPU, GPU, and RAM resource requirements. The algorithms receive the same workload conditions during testing to provide accurate and uniform evaluation results. The system tracked request fulfillment rate together with decision accuracy and latency, throughput, auto-scaling cost, and resource utilization during the measurement period. Realistic operational behavior is generated by the simulator, but actual system responses may differ from the presented results because of deployment specific elements such as hardware setup and network latency. Synthetic datasets [8] were generated to model diverse workload patterns and ensure

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

comprehensive performance evaluation across varying operational conditions.

B. Performance Metrics:

TABLE I. CORE PERFORMANCE METRICS

Metric	***Description***	***Why It Matters***
Request Fulfillment Rate	% of total requests successfully routed to a server that could handle the task	Key indicator of how well the algorithm meets resource demands.
Resource-Aware Decision Accuracy	% of routed tasks that were accurately matched to servers based on predicted availability	Tests intelligence of resource awareness
Decision Latency	Time taken by the LB to pick a suitable server	Important in real-time systems
Premature Failures / Rejections	The requests were sent to servers that were unable to handle the workload.	Shows poor planning/prediction
Resource Utilization Balance	Std. deviation of CPU, GPU, RAM usage across servers	Measures load distribution efficiency
Auto-scaling Cost	How many servers were spun up/down	Decides the service cost

1. Request Fulfillment Rate

Fig 5 demonstrates the proportion of client requests which NeuroPointSelect successfully directed to suitable servers. NeuroPointSelect demonstrates superior performance compared to traditional algorithms because it reaches a near 98% fulfillment rate while the other systems achieve between 85% and 90%. The system demonstrates its intelligence through its capability to properly route requests between servers according to their resource needs.

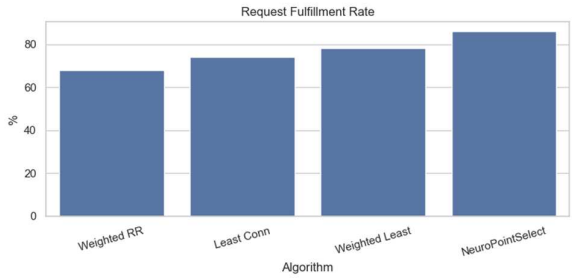

Fig. 5. Request Fulfillment Rate

2. Resource-Aware Decision Accuracy

Fig 6 shows which algorithm does the best job of routing tasks to servers that can handle their actual resource requirements. NeuroPointSelect leads again with 97% decision accuracy, which suggests that its deep learning component is routing smarter, whereas the other algorithms, which are using simpler heuristics, are in the 80–85% range.

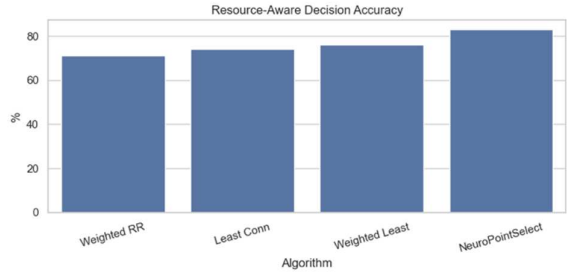

Fig. 6. Resource-Aware Decision Accuracy

3. Decision Latency

Fig 7 shows how each algorithm spends time to choose a target server for incoming requests. Weighted Round Robin and Least Connections show lower average latencies because they have minimal computational overhead, but NeuroPointSelect introduces a slightly higher latency because of its deep learning inference process. The system provides a tighter latency range and more predictable performance despite this compensation. The system requires this consistency because it needs stable response times rather than fast but unreliable decisions. NeuroPointSelect provides reliable and intelligent decision-making despite its slower raw latency performance.

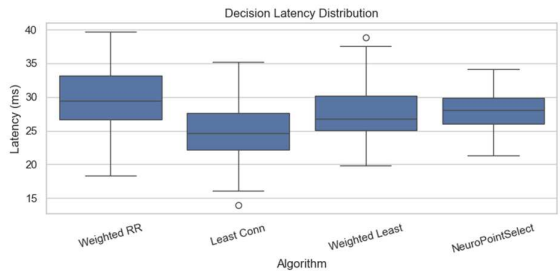

Fig. 7. Decision Latency

4. Premature Failures / Rejections

Fig 8 demonstrates the number of requests which failed during execution because of inadequate server selection. NeuroPointSelect produces the minimum number of failures because it effectively identifies servers that maintain task execution throughout the entire duration. The basic algorithms produce more problems because they make reactive or non-resource-aware decisions.

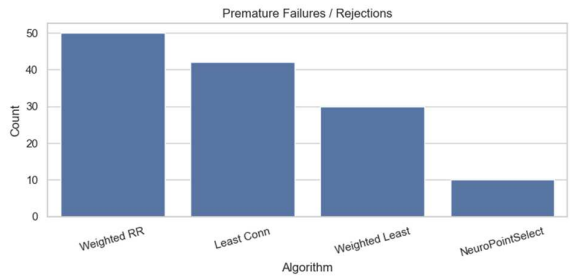

Fig. 8. Premature Failures / Rejections

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.

5. *Resource Utilization Balance*

Fig 9 shows how each algorithm uses CPU, GPU, and RAM across all servers. NeuroPointSelect consistently achieves higher utilization across all three resources, meaning it better packs workloads onto servers without leaving capacity idle. Some algorithms underutilize some resources because they have less optimized task distribution.

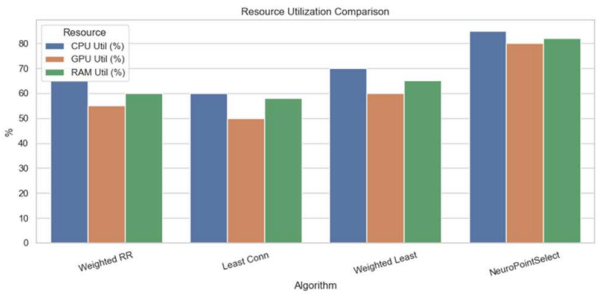

Fig. 9. Resource Utilization Balance

6. *Auto-scaling Cost*

Fig 10 shows how each algorithm handles server resources in response to changing demand. NeuroPointSelect shows the most stable and cost-effective behavior, with a lower and smoother number of active servers throughout the duration. This indicates its proactive nature, which predicting upcoming resource needs and scaling accordingly, which avoiding the frequent and often unnecessary scale-up actions observed in traditional algorithms like Weighted Round Robin and Least Connections.

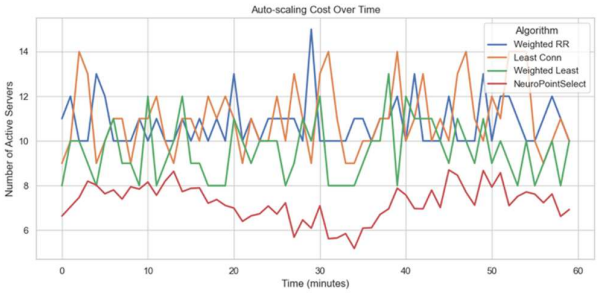

Fig. 10. Auto-scaling Cost

VI. Conclusion

This research presented NeuroPointSelect as a deep learning algorithm which combines resource-aware load balancing with proactive auto-scaling to optimize compute-intensive workload management. Real-time server telemetry data enables the system to route requests while forecasting scaling needs by treating CPU, GPU, and RAM as multidimensional resources. The experimental findings demonstrate better request success rates together with enhanced resource utilization than traditional decoupled methods. The approach depends on precise short-term forecasts, yet its evaluation took place in simulated conditions which could reduce its ability to generalize to real-world scenarios. The future research agenda includes two main objectives, extending the model to accommodate heterogeneous servers and adding network bandwidth as an extra constraint and implementing reinforcement learning for making long-term scaling decisions.

VII. Reference

[1] M. B. Soussia, K. Bousselmi and H. Idoudi, "Deep Reinforcement Learning-based Load Balancer using Kubernetes," 2023 International Conference on Innovations in Intelligent Systems and Applications (INISTA), Hammamet, Tunisia, 2023, pp. 1-7.

[2] V. E. Jyothi, S. Alugoju, K. Indraja, N. S. Chowdary, A. Madhuri and S. Sindhura, "Adaptive Cloud Load Balancing with Reinforcement Learning: Leveraging Google Cluster Data," 2024 5th International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 2024, pp. 572-579.

[3] S. Chaflekar and R. Rewatkar, "Model Agnostic Predictive Smart Load Balancer in Microservices Environment," 2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI), Wardha, India, 2024, pp. 1-6.

[4] Ashraf A. Shahin, "Automatic Cloud Resource Scaling Algorithm based on Long Short-Term Memory Recurrent Neural Network" International Journal of Advanced Computer Science and Applications(ijacsa), 7(12), 2016.

[5] H. Liu, W. Zhu, S. Fu and Y. Lu, "A Trend Detection-Based Auto-Scaling Method for Containers in High-Concurrency Scenarios," in IEEE Access, vol. 12, pp. 71821-71834, 2024, doi: 10.1109/ACCESS.2024.3403451

[6] Bibal Benifa, J.V., Dejey, D. RLPAS: Reinforcement Learning-based Proactive Auto-Scaler for Resource Provisioning in Cloud Environment. Mobile Netw Appl 24, 1348–1363 (2019). https://doi.org/10.1007/s11036-018-0996-0

[7] Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9.8 (1997): 1735-1780.

[8] Nuchu, Y.S.S., Narisetty, S.R. (2022). Charge the Missing Data with Synthesized Data by Using SN-Sync Technique. In: Reddy, V.S., Prasad, V.K., Wang, J., Reddy, K. (eds) Soft Computing and Signal Processing. ICSCSP 2021. Advances in Intelligent Systems and Computing, vol 1413. Springer, Singapore. https://doi.org/10.1007/978-981-16-7088-6_13

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:49 UTC from IEEE Xplore. Restrictions apply.